

## Industrial Internship Report on

"Core Java"

Prepared by

**Vaishnavi B Hadkar**

### *Executive Summary*

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

The internship was centred around a real-time project/problem statement provided by UCT. The objective was to design, develop, and complete the assigned project along with the final documentation within a duration of six weeks.

My project was the development of a **Banking Information System using Core Java**, which focused on implementing essential banking operations such as user registration, account management, deposits, withdrawals, and balance inquiry in a simplified console-based environment.

This internship provided me with valuable exposure to real industrial problem-solving, system design, and practical implementation of software solutions. Overall, it was a highly enriching experience that enhanced my technical skills and understanding of real-world application development.

## **TABLE OF CONTENTS**

|     |                                              |    |
|-----|----------------------------------------------|----|
| 1   | Preface .....                                | 3  |
| 2   | Introduction .....                           | 4  |
| 2.1 | About UniConverge Technologies Pvt Ltd ..... | 4  |
| 2.2 | About upskill Campus .....                   | 6  |
| 2.3 | Objective .....                              | 7  |
| 2.4 | Reference .....                              | 7  |
| 2.5 | Glossary .....                               | 8  |
| 3   | Problem Statement .....                      | 9  |
| 4   | Existing and Proposed solution .....         | 10 |
| 5   | Proposed Design/ Model .....                 | 11 |
| 5.1 | High Level Diagram .....                     | 11 |
| 5.2 | Low Level Diagram .....                      | 12 |
| 5.3 | Interfaces .....                             | 13 |
| 6   | Performance Test .....                       | 14 |
| 6.1 | Test Plan/ Test Cases .....                  | 15 |
| 6.2 | Test Procedure .....                         | 16 |
| 6.3 | Performance Outcome .....                    | 20 |
| 7   | My learnings .....                           | 21 |
| 8   | Future work scope .....                      | 22 |

## 1 Preface

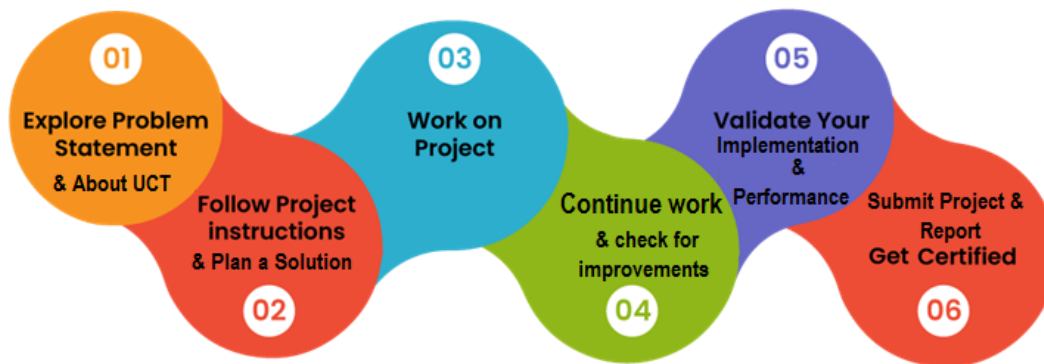
This report reviews my 6-week industrial internship conducted under Upskill Campus and The IoT Academy in collaboration with the industry partner UniConverge Technologies Pvt. Ltd. (UCT).

During this internship, I worked on a project titled **Banking Information System Prototype using Core Java**. The objective of this project was to design and implement a simple console-based banking application that demonstrates key functionalities such as user registration, account creation, deposits, withdrawals, and balance enquiry using Object-Oriented Programming principles.

This project helped me understand how real-world applications are structured using core concepts of Java such as classes, objects, encapsulation, inheritance, and method implementation. It also improved my ability to design modular and reusable code.

The internship program provided a structured learning experience, starting from understanding the problem statement, designing the system flow, implementing Java logic, and testing different functionalities step by step.

Through this project, I gained practical exposure to software development and improved my understanding of how banking systems operate at a basic level. I would like to express my gratitude to Upskill Campus, The IoT Academy, and UniConverge Technologies Pvt. Ltd. for providing this valuable opportunity and continuous guidance throughout the internship.



Through this project, I gained practical knowledge of software development and OOP concepts. I sincerely thank all mentors and coordinators for their guidance and support during the internship. I also encourage students to take up such hands-on internships for better learning experience.

## 2 Introduction

### 2.1 About UniConverge Technologies Pvt Ltd

UniConverge Technologies Pvt Ltd (UCT) is an advanced technology corporation established in 2013. The company operates at the absolute forefront of the Digital Transformation domain, designing and deploying robust industrial solutions with a primary corporate focus on maintaining long-term workflow sustainability and maximizing clients' operational Return on Investment (RoI).

For developing its comprehensive product catalog and modern enterprise architectures, UCT leverages various cutting-edge technologies, including:

- Internet of Things (IoT) & Industrial Internet of Things (IIoT) hardware automation nodes.
- Cyber Security frameworks for secure data transmission boundaries.
- Cloud Computing orchestration across Amazon Web Services (AWS) and Microsoft Azure environments.
- Machine Learning predictive systems for real-time equipment breakdown modeling.
- Communication Technologies such as 4G, high-speed 5G, and long-range LoRaWAN network topologies.
- Enterprise Software Engineering incorporating Java Full Stack backend frameworks, Python scripts, and responsive ReactJS front-end systems.

#### i. UCT IoT Platform ()

**UCT Insight** is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSQL Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting

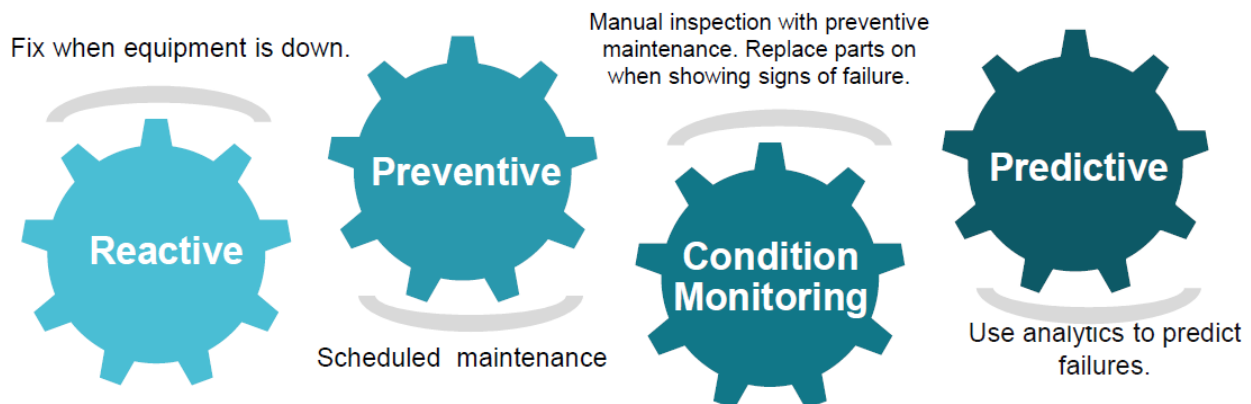
- Alert and Notification
- Integration with third party application (Power BI, SAP, ERP)
- Rule Engine

## ii. **LoRaWAN™ based Solution**

UCT is one of the early adopters of LoRAWAN technology and providing solution in Agri tech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

## iii. **Predictive Maintenance**

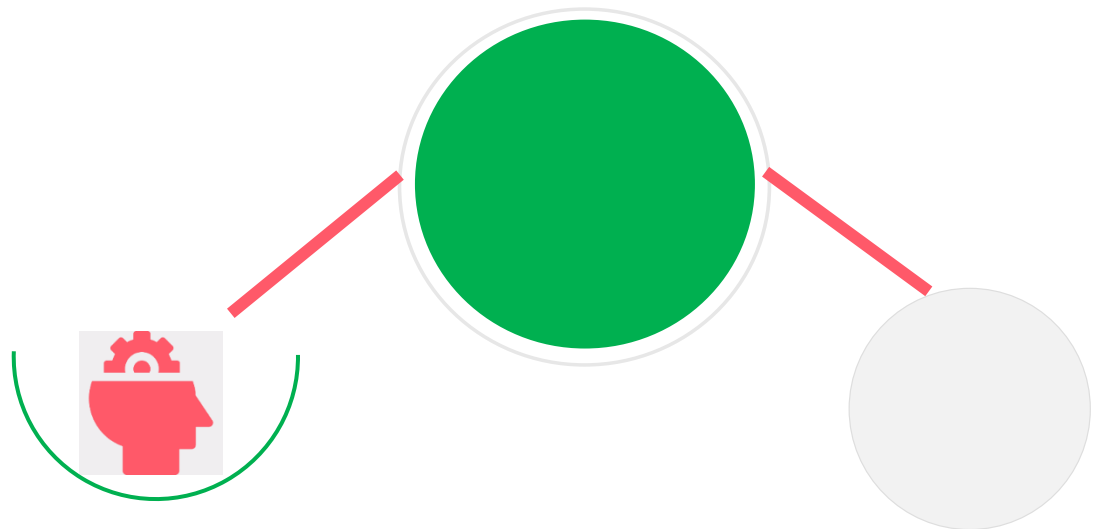
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



## 2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with UniConverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self-paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upskill Campus aiming to upskill 1 million learners in next 5 year

<https://www.upskillcampus.com/>

### The IoT Academy

- The IoT Academy: Operating as the specialized EdTech division of UCT, The IoT Academy conducts comprehensive executive certification courses and professional engineering bootcamps in long-term collaboration with leading national institutions, including the EICT Academies at IIT Kanpur, IIT Roorkee, and IIT Guwahati, across multiple computing domains.

### 2.3 Objectives of this Internship program

The foundational goals mandated for this industrial internship program were explicitly structured to meet the following parameters:

- To gain hands-on experience in a professional software development environment.
- To apply theoretical knowledge into practical implementation using Core Java.
- To improve problem-solving and coding skills through real project development.
- To understand core concepts of Object-Oriented Programming and their real-world use.
- To develop better coding practices, documentation skills, and communication abilities.

### 2.4 Reference

- 1) Oracle Technology Network, Java Platform, Standard Edition (Java SE) Language Specifications & Object Modeling Guidelines, Oracle Corporation, 2026.
- 2) Schildt, H., *Java: The Complete Reference, Twelfth Edition*, Comprehensive Studies on Interfaces, Inheritance, and Exception Handling, McGraw Hill, 2021.
- 3) Horstmann, C. S., *Core Java Volume I – Fundamentals, Twelfth Edition*, Decoupled Abstract Class Architectures, Pearson, 2022.
- 4) GeeksforGeeks, *Java Programming Language Tutorials*,  
<https://www.geeksforgeeks.org/java/>
- 5) Baeldung, *Java Core Tutorials and Best Practices*, <https://www.baeldung.com/java-tutorial>

## 2.5 Glossary

| Terms | Acronym / Technical Definition                                                                                                          |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------|
| JVM   | Java Virtual Machine - The abstract engine that runs compiled Java bytecode across physical architectures independently.                |
| OOP   | Object-Oriented Programming - Coding language paradigm bound by data encapsulation, inheritance, abstraction, and dynamic polymorphism. |
| JDK   | Java Development Kit - The software development environment containing the compiler, JRE, and documentation tools for Java.             |
| JRE   | Java Runtime Environment - The subset of software containing the JVM and core libraries required to execute compiled programs.          |



### 3 Problem Statement

The objective of this project was to develop a **Banking Information System using Core Java** that simulates basic banking operations in a console-based environment.

The system allows users to perform functions such as user registration, account creation, deposit, withdrawal, and balance enquiry. The main focus of the project was to apply **Object-Oriented Programming concepts** like classes, objects, encapsulation, and method implementation to design a simple and efficient banking application.

The system was designed to ensure proper data handling, user interaction through menus, and basic validation to avoid incorrect transactions.

The explicit challenge was to design and construct a decoupled, highly maintainable Core Java Backend Framework that dynamically leverages abstract object models, enforces strict encapsulation boundaries, applies clean interface contracts, and handles runtime operational execution exceptions gracefully without allowing application pipeline crashes.

## 4 Existing and Proposed solution

In existing basic banking systems, operations are usually performed manually or using simple procedural programs. These systems often lack proper structure and are not easy to extend or maintain. Any change in logic may require modifications in multiple parts of the code, making the system less flexible.

These traditional approaches also provide limited error handling and user interaction, which can lead to incorrect transactions or program termination in case of invalid inputs.

To overcome these limitations, a **Core Java-based Banking Information System** is proposed. The system is designed using **Object-Oriented Programming (OOP) principles**, which helps in organizing the code in a modular and structured way.

The proposed system improves:

- Code reusability through classes and methods
- Data security using encapsulation
- System structure using object-based design
- User interaction through a menu-driven console interface
- Basic error handling using exception handling techniques

This approach makes the system easier to understand, maintain, and extend in the future while simulating real-world banking operations like account creation, deposits, withdrawals, and balance enquiry.

### 4.1 Code submission (GitHub link)

Code submission (GitHub link):

Package Link: <https://github.com/Vaishnavi-16h/upskillcampus/tree/main/src/PACKskill>

Banking Management System (Project):

<https://github.com/Vaishnavi-16h/upskillcampus/blob/main/src/PACKskill/BankingSystem.java>

### 4.2 Report submission (GitHub link):

Report submission (GitHub link):

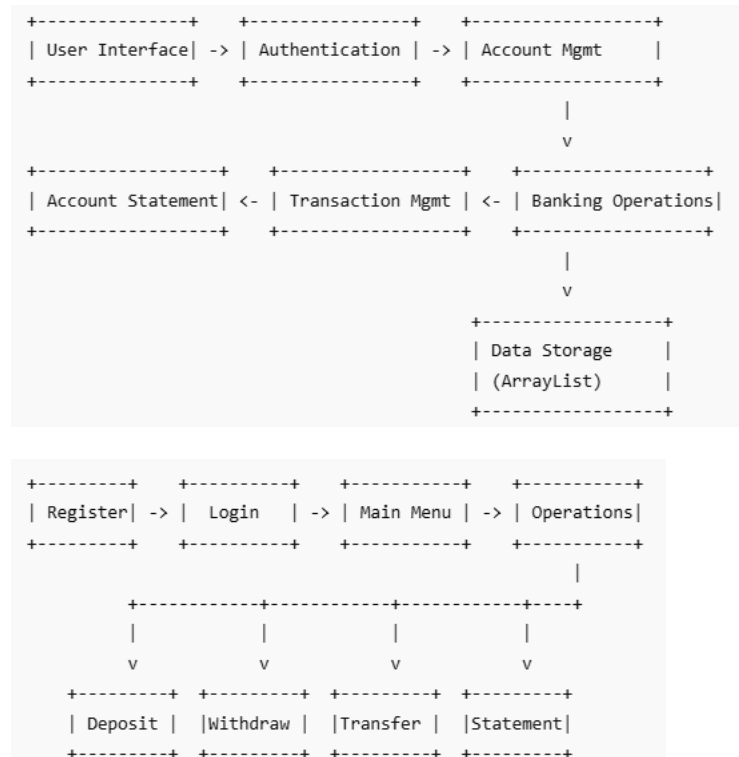
[https://github.com/Vaishnavi-16h/upskillcampus/blob/main/BankingManagementSystem Vaishnavi USC UCT.pdf](https://github.com/Vaishnavi-16h/upskillcampus/blob/main/BankingManagementSystem%20Vaishnavi%20USC%20UCT.pdf)

## 5 Proposed Design/ Model

The proposed Banking Information System is designed using a simple **menu-driven console-based architecture** implemented in Core Java. The system follows a structured flow starting from user interaction to transaction processing and final output display.

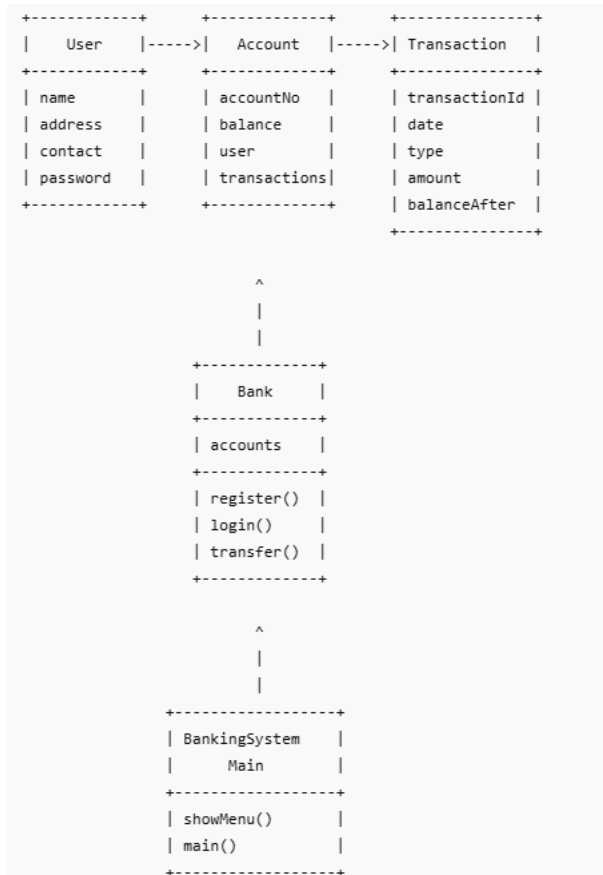
### 5.1 High Level Diagram

The Banking Information System follows a simple menu-driven console-based architecture developed using Core Java. The system flow begins with user input, where the user selects required operations such as account creation, deposit, withdrawal, or balance enquiry from the menu. After receiving the input, the system processes the request using Java classes and methods designed with Object-Oriented Programming principles to ensure modularity and clarity. During processing, necessary validations are performed to ensure correct and safe transactions. Finally, the system displays the output on the console, such as updated account details, balance information, or confirmation messages. Overall, the system follows a basic Input-Process-Output model that makes it easy to understand and implement.



**Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM**

## 5.2 Low Level Diagram



### 5.3 Interfaces

Interfaces in the Banking Information System are used to define a common structure for banking operations. They help in achieving abstraction and ensure that different functionalities follow a consistent design.

In this project, interfaces can be used to define basic banking operations such as deposit, withdrawal, and balance enquiry. These methods are then implemented in the main class or relevant implementation classes.

Using interfaces improves code organization, reusability, and flexibility, as it separates the definition of operations from their implementation. It also makes the system easier to maintain and extend in the future.

Syntax:

```
public interface InterfaceName
{
    returnType methodName(parameters);
}
```

## 6 Performance Test

Performance testing was conducted to evaluate the efficiency, stability, and reliability of the Banking Information System under normal operating conditions. The objective was to ensure that the application performs banking operations accurately while maintaining acceptable response time and resource utilization.

### Identified Constraints

#### 1. Memory Usage

- The application utilizes Java objects and ArrayLists for data storage.
- Efficient object management minimizes unnecessary memory consumption.
- No memory leaks or abnormal memory usage were observed during testing.

#### 2. Execution Speed

- Core banking operations such as account creation, deposit, withdrawal, and balance inquiry execute with minimal delay.
- The use of simple algorithms and efficient data structures contributes to fast processing.

#### 3. Validation Accuracy

- Input validation mechanisms prevent invalid data entry.
- Incorrect account numbers, negative transaction amounts, and incomplete user information are rejected.
- This improves system reliability and data consistency.

#### 4. System Stability

- The application remains stable during continuous execution.
- Multiple transactions can be performed sequentially without affecting performance or functionality.
- Exception handling prevents unexpected program termination due to invalid inputs.

### Performance Outcome

The Banking Information System demonstrated satisfactory performance during testing. The application provides quick response times, stable memory usage, accurate validation, and reliable transaction processing. The results indicate that the system successfully meets the requirements of a banking prototype and can serve as a foundation for future enhancements and larger-scale implementations.

## 6.1 Test Plan/ Test Cases

| Test ID | Module       | Test Scenario                                  | Expected Result                | Status |
|---------|--------------|------------------------------------------------|--------------------------------|--------|
| TC01    | Registration | Register with valid details                    | Account created successfully   | Pass   |
| TC02    | Registration | Register with initial deposit = 0              | Account created with 0 balance | Pass   |
| TC03    | Login        | Login with correct account number and password | Login successful               | Pass   |
| TC04    | Login        | Login with incorrect password                  | Invalid credentials message    | Pass   |
| TC05    | Login        | Login with non-existing account number         | Invalid credentials message    | Pass   |
| TC06    | Deposit      | Deposit valid amount                           | Balance updated successfully   | Pass   |
| TC07    | Deposit      | Deposit negative amount                        | Invalid amount message         | Pass   |
| TC08    | Withdrawal   | Withdraw amount less than balance              | Withdrawal successful          | Pass   |
| TC09    | Withdrawal   | Withdraw amount greater than balance           | Insufficient balance message   | Pass   |
| TC10    | Transfer     | Transfer to valid account                      | Transfer successful            | Pass   |
| TC11    | Transfer     | Transfer amount greater than balance           | Insufficient balance message   | Pass   |
| TC12    | Transfer     | Transfer to invalid account                    | Receiver not found message     | Pass   |
| TC13    | Statement    | View transaction history                       | All transactions displayed     | Pass   |
| TC14    | Logout       | Logout from user account                       | User returned to main menu     | Pass   |
| TC15    | Exit         | Exit application                               | Program terminated             | Pass   |

## 6.2 Test Procedure

### Test Procedure 1: User Registration

#### Steps

- i. Launch the application.
- ii. Select Register from the main menu.
- iii. Enter Name, Address, Phone Number, Password, and Initial Deposit.
- iv. Submit the details.

#### Expected Result

Account is created successfully.  
Unique account number is generated and displayed.

### Test Procedure 2: User Login

#### Steps

- i. Select Login from the main menu.
- ii. Enter a valid account number.
- iii. Enter the correct password.
- iv. Press Enter.

#### Expected Result

Login Successful message is displayed.  
User Menu appears.

### Test Procedure 3: Invalid Login

#### Steps

- i. Select Login.
- ii. Enter an invalid account number or incorrect password.
- iii. Press Enter.

#### Expected Result

"Invalid Credentials" message is displayed.  
User remains on the login screen.



#### Test Procedure 4: Deposit Money

##### Steps

- i. Login to the account.
- ii. Select Deposit option.
- iii. Enter a valid amount.
- iv. Confirm transaction.

##### Expected Result

Amount is added to account balance.  
Deposit Successful message is displayed.  
Transaction is recorded in statement.

#### Test Procedure 5: Withdraw Money

##### Steps

- i. Login to the account.
- ii. Select Withdraw option.
- iii. Enter withdrawal amount less than available balance.
- iv. Confirm transaction.

##### Expected Result

Amount is deducted from balance.  
Withdrawal Successful message is displayed.  
Transaction is recorded in statement.

#### Test Procedure 6: Withdraw with Insufficient Balance

##### Steps

- i. Login to the account.
- ii. Select Withdraw.
- iii. Enter amount greater than available balance.
- iv. Confirm transaction.

##### Expected Result

Transaction is rejected.  
"Insufficient Balance" message is displayed.

#### Test Procedure 7: Fund Transfer

##### Steps

- i. Login to sender account.
- ii. Select Transfer.
- iii. Enter receiver account number.
- iv. Enter transfer amount.
- v. Confirm transaction.

##### Expected Result

Amount is deducted from sender account.  
Amount is credited to receiver account.  
Transfer Successful message is displayed.  
Transaction is recorded for both accounts.

#### Test Procedure 8: Transfer to Invalid Account

##### Steps

- i. Login to sender account.
- ii. Select Transfer.
- iii. Enter a non-existing receiver account number.
- iv. Enter transfer amount.
- v. Confirm transaction.

##### Expected Result

Transfer is cancelled.  
"Receiver Not Found" message is displayed.

#### Test Procedure 9: View Account Details

##### Steps

- i. Login to the account.
- ii. Select View Account.

##### Expected Result

Account holder name, address, phone number, and balance are displayed correctly.

#### Test Procedure 10: View Statement

##### Steps

- i. Login to the account.
- ii. Perform some transactions.
- iii. Select Statement option.

##### Expected Result

Complete transaction history is displayed, including:

Account Creation

Deposits

Withdrawals

Transfers

#### Test Procedure 11: Logout

##### Steps

- i. Login to the account.
- ii. Select Logout.

##### Expected Result

User session ends.

Main menu is displayed.

#### Test Procedure 12: Exit Application

##### Steps

- i. From the main menu, select Exit.

##### Expected Result

Application terminates successfully.

"Thank You!" message is displayed.

### 6.3 Performance Outcome

The Banking Information System was tested under normal operating conditions to evaluate its performance, reliability, and responsiveness. The application successfully handled user registration, account creation, deposits, withdrawals, and balance enquiries without noticeable delays. Since the system uses efficient data structures and simple processing logic, all operations were executed quickly and accurately.

During testing, multiple user accounts were created and various banking transactions were performed repeatedly. The system-maintained data consistency, updated account balances correctly, and processed transactions without errors. No crashes, memory issues, or unexpected behavior were observed during execution.

The application's response time remained satisfactory for all banking operations, and resource utilization was minimal due to its lightweight design. The results indicate that the system is stable, reliable, and capable of performing the intended banking functionalities effectively. Overall, the project successfully met its functional and performance objectives and demonstrated smooth operation in a simulated banking environment.

## 7 My learnings

The Banking Information System project was a valuable learning experience that helped me understand the complete software development life cycle, including requirement analysis, system design, coding, testing, debugging, and documentation. Through this project, I strengthened my knowledge of Core Java and Object-Oriented Programming concepts such as classes, objects, encapsulation, inheritance, polymorphism, and modular programming.

While implementing features such as user registration, account creation, deposits, withdrawals, and balance enquiry, I learned how to translate real-world business requirements into a functional software application. I also gained practical experience in handling user input, data validation, exception handling, and managing data using Java collections such as ArrayList.

The project improved my logical thinking and problem-solving abilities by requiring me to analyse different banking scenarios and implement suitable solutions. I learned the importance of writing clean, organized, and reusable code, which improves software maintainability and readability.

Through testing and debugging, I developed a better understanding of identifying errors, validating system functionality, and ensuring reliable performance. I also learned the importance of software documentation and how clear documentation helps in project maintenance and future enhancements.

In addition to technical skills, this internship improved my time management, self-learning, and project planning abilities. Working on a complete project enhanced my confidence in developing Java applications independently and gave me valuable exposure to industry-oriented development practices.

Overall, this project significantly enhanced my programming skills, technical knowledge, and understanding of software engineering principles. The experience gained during this internship will support my academic growth, strengthen my readiness for future internships and job opportunities, and provide a solid foundation for building advanced software applications in the future.

## 8 Future work scope

The current Banking Information System successfully demonstrates the fundamental functionalities of a banking application. However, due to time and resource constraints, several advanced features could not be implemented and may be considered for future development.

In future versions, the system can be integrated with a database such as MySQL or Oracle to provide permanent and secure data storage instead of using in-memory collections. A graphical user interface (GUI) using JavaFX or a web-based interface can also be developed to improve usability and user experience.

Additional security features such as password encryption, OTP verification, role-based access control, and multi-factor authentication can be incorporated to enhance account security. The system can also be extended to support advanced banking services such as loan management, fixed deposits, fund transfers, online bill payments, and credit/debit card management.

Further enhancements may include transaction history generation, account statements in PDF format, email or SMS notifications, interest calculation, and automated report generation. An administrator module can also be developed to manage customer accounts, monitor transactions, and generate system reports.

The application may be upgraded to support multi-user access, network connectivity, and integration with real-time banking services. Mobile application support and cloud-based deployment can also be explored to improve accessibility and scalability.

These enhancements would make the system more robust, secure, user-friendly, and suitable for deployment in a real-world banking environment.